



API PROTOCOL DOCUMENT

04.04.2025

CONTENT

1. Revise Record	2
2. Overview	4
3. Keywords	4
4. BLE Keywords	4
5. Wi-Fi Keywords	4
6. Peripheral and Central Devices Through BLE	5
6.1 Scan	5
6.2 Connect	5
6.3 Service Discovery	5
6.4 Handshake	5
6.5 Pair	5
7. Connect the Peripheral Device to Wi-fi	6
8. Device and Server Through HTTP	7
9. Keywords	11

REVISE RECORD

Version	Description	Date	Author	Authorizer
v1.0		04.04.2025	Hayriye Kaymaz	Faik Baharoğlu

Overview

This document defines the communication protocol between the Onloi Smart Lock System's devices and its central server using Bluetooth Low Energy (BLE) and HTTP communication.

Keywords

- **DeviceId:** deviceId is a parameter that represents the unique identity of the device.
- **pairKey:** Pair Key and device key are generated during device pairing using Bluetooth BLE.
- **macAddress:** The device's MAC Address. s
- **rsi:** This stands for Received Signal Strength Indicator, which indicates the signal strength of a wireless network.
- **frequency:** This refers to the frequency band (e.g., 2.4 GHz or 5 GHz) used by a wireless network.
- **name:** This refers to the name of the wireless network (SSID).

BLE Keywords

Values that the **Command** can take:

- **Handshake:** It refers to a type of handshake or authentication process performed during connection establishment.
- **Pair:** This command is used to establish a secure connection between the central device and the peripheral device.
- **getSsid:** This refers to the process of retrieving the list of available SSIDs (Service Set Identifiers).
- **connectWifi:** It is used to check whether the WiFi connection is established or not.

Wi-Fi Keywords

Values that the **Command** can take:

- **log:** When logged in, it can take 3 values: RFID, Keypad and FingerPrint.
- **pass:** Pass is triggered, it takes a number with a digit count between 4 and 10.
- **state:** When the command is "state" the data will return as "wifireset" and also provide the current time.
- **open:** When the operation is successful, the HTTP status value is returned.



Peripheral and Central Devices Through BLE



Scan:

Keep the peripheral device discoverable to scan. The device must only be discoverable during initial pairing or after a factory reset by the user. Broadcast device name as “Onloi Lock”, then the central device will be able to recognize this device as an Onloi device.



Connect:

Connections will be applied automatically between both devices; no preparation is required on your part.



Service Discovery:

Once connected, the Central queries the peripheral's services and characteristics. The peripheral device is responsible to send services and characteristics information to the central device just after the connection is performed.



Handshake:

1- The central device will write the following JSON on a characteristic of the peripheral device which is received on connection establishment.

```
{
  "command": "handshake"
}
```

2- The peripheral device will send the following JSON on a characteristic as the response to the central device.

```
{
  "command": "handshake",
  "deviceId": "string",
  "createdAt": <timestamp>
}
```



Pair:

1- The central device will write the following JSON on a characteristic of the peripheral device. The peripheral device should save “deviceKey” and “pairKey” for further Http requests.

```
{
  "command": "pair",
  "deviceKey": "string",
  "pairKey": "string"
}
```

2- The peripheral device will send the following JSON on a characteristic as the response to the central device.

```
{
  "command": "pair",
  "result": "succeeded" | "failed"
}
```

Connect the Peripheral Device to Wi-fi

1- The central device will write the following JSON on a characteristic of the peripheral device.

```
{
  "command": "getSsids"
}
```

2- The peripheral device will send the following JSON on a characteristic as the response to the central device.

```
{
  "command": "getSsids",
  "result": {
    "0": {
      "macAddress": "string",
      "rssi": <number>,
      "frequency": 2.4 | 5 | 6,
      "name": "string"
    }
  }
}
```

- 3- The central device will write the following JSON on a characteristic of the peripheral device.

```
{
  "command": "connectWifi",
  "ssid": "string",
  "macAddress": "string",
  "pass": "string"
}
```

- 4- The peripheral device will send the following JSON on a characteristic as the response to the central device.

```
{
  "command": "connectWifi",
  "result": "succeeded" | "failed"
}
```

Attention:

- Data should be sent in chunks. Each packet can contain up to 20 bytes. Separate each message with a zero byte.

Device and Server Through HTTP

Request:

```
Method: POST
URL: https://onloi.universeio.com/nowhere
Content-Type: application/json
```

- Body Format:

```
{
  "action": "run",
  "name": "device",
  "args": {
    "command": "open",
    "pairKey": "<string>",
    "deviceKey": "<string>",
    "deviceId": "<string>",
    "data": "openlock",
    "createdAt": <number timestamp in milliseconds>
  }
}
```

➔ Example:

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{
    "action": "run",
    "name": "device",
    "args": {
      "command": "open",
      "pairKey": "s987y7x6cv4s5gfd579",
      "deviceKey": "m7n8wg5r-cl8s",
      "deviceId": "78356203936",
      "createdAt": 1740751085225
    }
  }' https://onloi.universeio.com/nowhere
```

Responses:

- Open the Lock:

```
HTTP Status: 200
Body: 'ok'
```

- Don't Open the Lock:

HTTP Status: 403
Body: 'forbidden'

Request:

Method: POST
URL: <https://onloi.universeio.com/nowhere>
Content-Type: application/json

- Body Format:

```
{
  "action": "run",
  "name": "device",
  "args": {
    "command": "log",
    "pairKey": "string",
    "deviceKey": "string",
    "deviceId": "string",
    "data": "RFID1",
    "createdAt": <number timestamp in milliseconds>
  }
}
```

➔ Examples:

● Example 1 – Log with RFID1:

```
curl --header "Content-Type: application/json" \  
  --request POST \  
  --data '{  
    "action": "run",  
    "name": "device",  
    "args": {  
      "command": "log",  
      "pairKey": "s987y7x6cv4s5gfd579",  
      "deviceKey": "m7n8wg5r-cl8s",  
      "deviceId": "78356203936",  
      "data": "RFID1",  
      "createdAt": 1740751085225  
    }  
  }' https://onloi.universeio.com/nowhere
```

● Example 2 – Log with Keypad1:

```
curl --header "Content-Type: application/json" \  
  --request POST \  
  --data '{  
    "action": "run",  
    "name": "device",  
    "args": {  
      "command": "log",  
      "pairKey": "s987y7x6cv4s5gfd579",  
      "deviceKey": "m7n8wg5r-cl8s",  
      "deviceId": "78356203936",  
      "data": "Keypad1",  
      "createdAt": 1740751085225  
    }  
  }' https://onloi.universeio.com/nowhere
```

Example 3 – Log with FingerPrint1:

```
curl --header "Content-Type: application/json" \  
  --request POST \  
  --data '{  
    "action": "run",  
    "name": "device",  
    "args": {  
      "command": "log",  
      "pairKey": "s987y7x6cv4s5gfd579",  
      "deviceKey": "m7n8wg5r-cl8s",  
      "deviceId": "78356203936",  
      "data": "FingerPrint1",  
      "createdAt": 1740751085225  
    }  
  }' https://onloi.universeio.com/nowhere
```



Keywords

- **RFID:** RFID (Radio Frequency Identification) is a technology that enables the identification of objects, people, or animals using radio waves.
- **Keypad:** A keypad, also known as a keyset, is an input device that contains a series of keys.
- **FingerPrint:** FingerPrint is the unique pattern of ridges and valleys on the fingertip of a person. The second parameter in Data represents the ID of the connected device.

- Log Record:

HTTP Status: 200
Body: 'ok'

- Log Record:

HTTP Status: 403
Body: 'forbidden'

Request:

```
Method: POST
URL: https://onloi.universeio.com/nowhere
Content-Type: application/json
```

- Body Format:

```
{
  "action": "run",
  "name": "device",
  "args": {
    "command": "pass",
    "pairKey": "<string>",
    "deviceKey": "<string>",
    "deviceId": "<string>",
    "data": 2838901,
    "createdAt": <number timestamp in milliseconds>
  }
}
```

➔ Example 1:

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{
    "action": "run",
    "name": "device",
    "args": {
      "command": "pass",
      "pairKey": "s987y7x6cv4s5gfd579",
      "deviceKey": "m7n8wg5r-cl8s",
      "deviceId": "78356203936",
      "data": "2838901",
      "createdAt": 1740751085225
    }
  }' \
  https://onloi.universeio.com/nowhere
```

Attention:

- **data:** This value can range between 4 and 10 characters.

Responses:

- Open the Pass:

HTTP Status: 200

Body: 'ok'

- Don't Open the Pass:

HTTP Status: 403

Body: 'forbidden'

Request:

Method: POST

URL: <https://onloi.universeio.com/nowhere>

Content-Type: application/json

- Body Format:

```
Body:
{
  "action": "run",
  "name": "device",
  "args": {
    "command": "state",
    "pairKey": "string",
    "deviceKey": "string",
    "deviceId": "string",
    "data": "getdata",
    "createdAt": <number timestamp in milliseconds>
  }
}
```

➔ Example 1:

```
curl --header "Content-Type: application/json" \  
  --request POST \  
  --data '{  
    "action": "run",  
    "name": "device",  
    "args": {  
      "command": "state",  
      "pairKey": "s987y7x6cv4s5gfd579",  
      "deviceKey": "m7n8wg5r-cl8s",  
      "deviceId": "78356203936",  
      "data": "wifireset",  
      "createdAt": 1740751085225  
    }  
  }' \  
  https://onloi.universeio.com/nowhere
```

Attention:

- **getdata:** This value takes the Wi-Fi reset and the current time. (wifireset, time)

Responses:

- **Open the Wifireset:**

```
HTTP Status: 200  
Body: 'ok'
```

- **Don't Open the Wifireset:**

```
HTTP Status: 403  
Body: 'forbidden'
```

- **Time:**

```
HTTP Status: 200  
Body: 'ok'
```

- **Time:**

HTTP Status: 403

Body: 'forbidden'